Quotients, remainders, and encoded computations

A companion to Section 4 of the BB(3) decider paper

1. One equation, two uses

For natural numbers N and $d > 0$, division with remainder says

$$N = ad + v, \qquad v < d.$$

Here a is the quotient and v is the remainder. All variables range over natural numbers, so $v \geq 0$ is automatic. Examples are

$$17 = 5 \cdot 3 + 2, \qquad 12 = 3 \cdot 4 + 0, \qquad 2 = 0 \cdot 5 + 2.$$

The last example matters: the divisor may exceed the number, and the quotient may be zero. The inequality $v < d$ is essential. Without it, both $17 = 5 \cdot 3 + 2$ and $17 = 4 \cdot 3 + 5$ would be allowed. With it, the quotient and remainder are unique. If two allowed decompositions had $a < a'$, then $ad + v < ad + d \leq a'd \leq a'd + v'$, contradicting their equality.

This is a checking equation, not an instruction to guess forever. An ordinary program computes $a = \lfloor N/d \rfloor$ and $v = N \bmod d$. Arithmetic describes the result using multiplication, addition, equality, and order. The notation mod need not be a primitive symbol of the theory.

Use 1: moving the machine head. A configuration is (q, L, b, R): state, left tape stack, scanned bit, right tape stack. The nearest cell on either side is the least significant bit. Thus $R = 7$ means the cells to the right are $1, 1, 1, 0, 0, \ldots$, nearest first. To move right after writing w:

$$L' = 2L + w, \qquad R = 2R' + b', \qquad b' < 2.$$

The equation pops the nearest right bit. For $R = 7$, it gives $R' = 3, b' = 1$, because $7 = 2 \cdot 3 + 1$. For $R = 0$, it gives $R' = 0, b' = 0$. The full executable update is

$$(q, L, b, R) \longmapsto (q', , 2L + w, , R \bmod 2, , \lfloor R/2 \rfloor).$$

Moving left swaps the roles of L and R: $(q', \lfloor L/2 \rfloor, L \bmod 2, 2R + w)$.

Use 2: reading a recorded computation. We also use remainders to encode a *whole sequence* of configurations. Here the divisor changes with the time index. This second use is the subject of the next four pages. Do not confuse a tape-stack number with a number encoding its successive values.

2. Encoding a column of a computation table

A finite computation is a table with four columns:

time i	q_i	L_i	b_i	R_i
0	q_0	L_0	b_0	R_0
1	q_1	L_1	b_1	R_1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	q_n	L_n	b_n	R_n

First-order arithmetic has number variables, not primitive list variables. For each column j, two numbers B_j, C_j encode its entries. Set

$$M_j(i) = 1 + (i+1)C_j, \qquad \beta_j(i,v) \equiv \exists a < H\,[B_j = aM_j(i) + v \ \wedge \ v < M_j(i)].$$

In words: *at time i, column j has value v if v is the remainder of B_j upon division by $M_j(i)$, with a quotient below H.*

Symbol	Role
i	Time index: which row we are reading
j	Column index: $0 = q$, $1 = L$, $2 = b$, $3 = R$
B_j	Number whose remainders store that column
C_j	Number determining its sequence of divisors
v	Proposed entry in the selected row and column
a	Auxiliary quotient, quantified inside β_j
H	Common bound on quotients and configuration fields
n	Proposed final time: n transitions, $n+1$ rows

The letters B_j have no connection with machine state B. The index j is explanatory notation selecting one of four formulas; it is not another history parameter.

Small example. Choose $B = 5, C = 2, H = 6$. Then

i	$M(i)$	division equation	decoded value
0	3	$5 = 1 \cdot 3 + 2$	2
1	5	$5 = 1 \cdot 5 + 0$	0
2	7	$5 = 0 \cdot 7 + 5$	5

So the pair encodes the entries $2, 0, 5$. In particular, $\beta(0,2)$ holds and $\beta(0,1)$ does not. If $H = 1$, the first entry cannot be decoded: its required quotient is 1, which fails $a < H$. Too small a bound can spoil a proposed code. It is therefore quantified along with the coding numbers.

The whole row is decoded by

$$\mathrm{At}(i,(q,L,b,R)) \equiv \beta_0(i,q) \wedge \beta_1(i,L) \wedge \beta_2(i,b) \wedge \beta_3(i,R).$$

Encoding does not certify execution. Any coding numbers specify remainders, but they may describe wrong states, a wrong starting tape, or impossible transitions. A separate formula checks these conditions.

3. An actual code for a halting machine

Let X be `1RB1LB_1LA___`. Its instructions are: $A0 \mapsto 1RB$, $A1 \mapsto 1LB$, $B0 \mapsto 1LA$, and $B1$ is missing. We start blank, with $A = 0$, $B = 1$. To match the paper, take $Z = 3$ and *replace the missing instruction by write 0, move left, enter Z*. The unused third working state's instructions may be chosen to enter Z too. In the paper's instruction numbering, the chosen $B1$ replacement has code 12.

i	Head	q	L	b	R	Instruction used next
0	0	0	0	0	0	$1RB$
1	1	1	1	0	0	$1LA$
2	0	0	0	1	1	$1LB$
3	−1	1	0	0	3	$1LA$
4	−2	0	0	0	7	$1RB$
5	−1	1	1	1	3	Enter Z by the chosen replacement
6	−2	3	0	1	6	Already halted

The head column is explanatory; it is not encoded. At time 5, $L = 1$ encodes the 1 at position -2, $b = 1$ is at -1, and $R = 3$ encodes ones at 0 and 1. The terminal replacement erases the cell at -1 and moves left: $L = 1 = 2 \cdot 0 + 1$ and $R' = 2 \cdot 3 + 0 = 6$. Thus the original missing-instruction machine stops after five ordinary moves; the paper's representation reaches Z after six transitions.

Choose $n = 6$ and $C_0 = C_1 = C_2 = C_3 = 60$. The seven divisors are

$$61,\ 121,\ 181,\ 241,\ 301,\ 361,\ 421.$$

They are pairwise coprime, and all exceed every entry in the table. The Chinese remainder theorem lets us choose a number with the specified remainders for each column. Here are actual choices:

Column	B_j	Remainders, at times $0, \ldots, 6$
q	8 853 985 391 702 276	0, 1, 0, 1, 0, 1, 3
L	3 710 433 271 215 804	0, 1, 0, 0, 0, 1, 0
b	13 837 268 216 790 557	0, 0, 1, 0, 0, 1, 1
R	12 184 268 548 412 037	0, 0, 1, 3, 7, 3, 6

Take $H = 13\,837\,268\,216\,790\,558$, one more than the largest B_j. Every decoded field and required quotient is then below H.

For example, reading the state at time 5 uses divisor 361:

$$8\,853\,985\,391\,702\,276 = 24\,526\,275\,323\,275 \cdot 361 + 1.$$

Reading the right stack at time 6 uses divisor 421:

$$12\,184\,268\,548\,412\,037 = 28\,941\,255\,459\,411 \cdot 421 + 6.$$

The numbers are large because this is a convenient general-purpose coding, not a compact file format. No optimality is claimed.

4. What exactly does "X halts" assert?

Let $\mathrm{Run}_X(n, H, B_0, C_0, \ldots, B_3, C_3)$ be the conjunction of four checks:

$$\begin{aligned}
\text{Rows exist:} &\quad \forall i < n+1\ \exists c < H\ \ \mathrm{At}(i, c);\\
\text{Start blank:} &\quad \forall c < H\,[\mathrm{At}(0, c) \to c = (0,0,0,0)];\\
\text{Follow } X\text{:} &\quad \forall i < n\ \forall c, e < H\,[(\mathrm{At}(i, c) \wedge \mathrm{At}(i+1, e)) \to \mathrm{Step}_X(c, e)];\\
\text{End halted:} &\quad \exists c < H\,[\mathrm{At}(n, c) \wedge q(c) = 3].
\end{aligned}$$

Here $c < H$ abbreviates four inequalities, and $\exists c < H$ abbreviates four bounded number quantifiers. The expressions $q(c)$ and tuple equality are shorthand for the relevant fields. Step_X expands to arithmetic equations for X's fixed instruction table; it is not an assumed oracle.

Why check row existence separately? The transition clause is an implication. If there were no decoded row, its antecedent could always be false and the clause would hold vacuously. The first clause prevents this. Remainder uniqueness also prevents one index from decoding two different rows.

For the numbers on page 3, the first check decodes all seven rows; the second gives the blank row at time 0. Each adjacent pair satisfies a step equation. For example, times 4 and 5 give

$$(0,0,0,7) \longmapsto (1,1,1,3), \qquad 1 = 2 \cdot 0 + 1, \quad 7 = 2 \cdot 3 + 1.$$

Finally, the row at time 6 has state 3. Therefore those particular ten numbers satisfy Run_X.

The halting sentence is

$$\exists n\, \exists H\, \exists B_0\, \exists C_0 \cdots \exists B_3\, \exists C_3 \quad \mathrm{Run}_X(n, H, B_0, C_0, \ldots, B_3, C_3).$$

The existential quantifiers mean that *some* numerical certificate passes all four checks. The page-3 values are witnesses. They do not have to be the only witnesses: enlarging H also works.

Examples of rejected claims. Keep the same codes but claim $n = 5$: the final decoded state is 1, not 3, so the final check fails. Alternatively, take $n = 0, H = 1$ and all $B_j = C_j = 0$. The blank row decodes with quotient 0, but it is not halted. Large coding numbers alone do not prove halting.

Here is a subtler false claim: take $n = 1, H = 4$, all $C_j = 2$, and $(B_0, B_1, B_2, B_3) = (3, 0, 0, 0)$. Divisors 3 and 5 decode the rows $(0,0,0,0)$ and $(3,0,0,0)$. The start and end checks pass, but the transition check fails: X cannot jump directly from blank to Z.

All quantifiers *inside* Run_X have explicit bounds. Consequently Run_X is a Δ_0 formula in the arithmetic language $\{0, 1, +, \cdot, <, =\}$. The outer existential quantifiers are unbounded, so the displayed halting sentence is Σ_1, not Δ_0. For this fixed example, proving halting requires checking concrete finite arithmetic and introducing existential witnesses; it does not require a theorem constructing codes for every possible computation length.

5. What exactly does "machine Y does not halt" assert?

In the same encoding the sentence is

$$\forall n\, \forall H\, \forall B_0\, \forall C_0 \cdots \forall B_3\, \forall C_3 \quad \neg\, \mathrm{Run}_Y(n, H, B_0, C_0, \ldots, B_3, C_3).$$

It excludes *every* candidate code, not just the examples we happened to simulate. Its unbounded universal quantifiers followed by a bounded formula make it Π_1. Classical logic identifies it with the negation of the halting sentence.

For a concrete Y, use the cycler `1RB0RB_0LA___`. From blank it repeats these four configurations:

$$\underbrace{(0,0,0,0)}_{d_0} \to \underbrace{(1,1,0,0)}_{d_1} \to \underbrace{(0,0,1,0)}_{d_2} \to \underbrace{(1,0,0,0)}_{d_3} \to d_0.$$

For instance, d_1 uses $B0 = 0LA$: it pushes 0 onto the right stack and pops $L = 1 = 2 \cdot 0 + 1$, producing d_2. The other three edges are checked by the same equations. None of these four configurations uses the missing $B1$ instruction. Define the quantifier-free invariant

$$I(c) \equiv (c = d_0 \vee c = d_1 \vee c = d_2 \vee c = d_3).$$

Finite arithmetic proves that blank satisfies I, that every allowed step from a configuration satisfying I again satisfies I, and that I excludes state 3. These are local facts about four rows, not assumptions that an infinite run already exists.

How bounded induction uses the history code. Fix arbitrary values of the ten history parameters and assume Run_Y. Induct on

$$\psi(i) \equiv [i < n + 1 \;\to\; \forall c < H\, (\mathrm{At}(i, c) \to I(c))].$$

The parameters n, H, B_j, C_j remain fixed throughout induction. Every quantifier in ψ is bounded, so this is a permitted $\mathrm{I}\Delta_0$ induction formula. At time 0, the initialization check gives I. For a successor time within the history, row existence supplies a predecessor; the transition check and invariant preservation give I for the next row. Outside the history the implication is vacuous. Induction therefore gives I at time n, contradicting the final check's state 3. Discharge the assumption Run_Y and universally quantify its arbitrary parameters.

The important limitation. This proof rejects supplied halting histories. It does not construct a history for every length. Over the usual natural numbers, every finite computation can be remainder-coded by an appropriate choice of divisors and a sufficiently large bound. A uniform internal coding-existence theorem, or equivalence with another halting encoding inside $\mathrm{I}\Delta_0$, is additional work, not supplied by this argument.

Checking the numerical example. For each page-3 B_j, divide by $1 + (i + 1)60$ for $i = 0, \ldots, 6$. The remainders must equal its listed column. For background and the exact conventions, see Sections 3–6 of *Busy Beaver decider certificates in bounded arithmetic* (the accompanying BB(3) paper). The calculations here are explanatory worked examples, not a new machine-checked Metamath submission.